Elaborazione di segnali e immagini

Mattia Nicolis Matteo Drago

A.A. 2025-26

Indice

Teoria **5**
Sistemi e segnali 5
Classificazione dei segnali 6
Segnali a tempo continuo e a tempo discreto 7

Laboratorio **9**
Introduzione a Matlab 9
Layout di Matlab 9
Definizione di matrici e vettori 10
Operazioni tra matrici 11
Comandi utili e istruzioni dos-like 12
Script e function 13
Istruzioni di controllo 14

Teoria

Sistemi e segnali

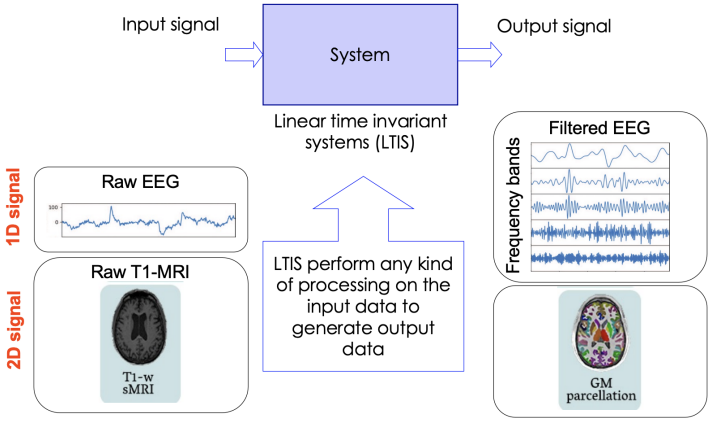


Figura 1: sistema e segnali

Definizione - segnale

Un **segnale** è un insieme di dati o informazioni.

Un segnale rappresenta qualsiasi tipo di variabile fisica soggetta a variazioni.

I segnali sono funzioni della variabile indipendente *tempo* (t) o *spazio* ($x, x = [x_1, x_2], x = [x_1, x_2, x_3]$).

Sia la variabile indipendente che la variabile fisica possono essere scalari o vettoriali.

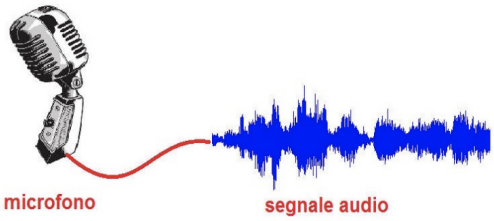
Esempi di segnali:

segnali 1D : segnale elettrocardiografico (EEG), segnale vocale, segnale audio

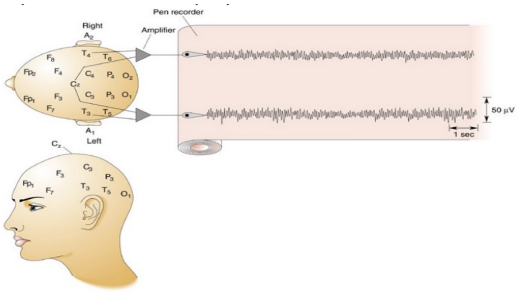
segnali 2D : segnale immagine, segnale video

segnali 3D : segnale di dati volumetrici

Un segnale può essere elaborato da un **sistema**.



(a) segnale 1D - segnale audio



(b) segnale 1D - elettrocardiogramma

Definizione - Sistema

Un **sistema** è un'entità che elabora un insieme di segnali (*input*) per produrre un altro insieme di segnali (*output*).

I sistemi processano i segnali per:

- estrarre informazioni
- consentire la trasmissione su canali con capacità limitata (JPEG, JPEG2000, codifica MPEG)
- migliorare la sicurezza sulle reti (crittografia, watermarking)
- migliorare la sicurezza sulle reti (crittografia, watermarking)

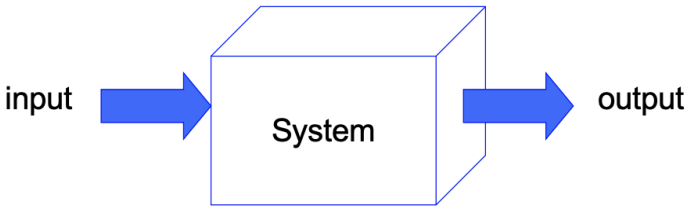


Figura 3: sistema

La funzione che collega l'uscita del sistema con il segnale di ingresso è chiamata **risposta all'impulso** $h(t)$ nel dominio del *tempo* e **funzione di trasferimento** $H(\omega)$ nel dominio della *frequenza*.

Classificazione dei segnali

I segnali possono essere di 4 tipologie:

- **segnali a tempo continuo** e **a tempo discreto**
- **segnali analogici** e **digitali**
- **segnali periodici** e **aperiodici**
- **segnali energetici** e **segnali di potenza**

- **segnali deterministici** e **probabilistci**

Segnali a tempo continuo e a tempo discreto

Laboratorio

Introduzione a Matlab

Layout di Matlab

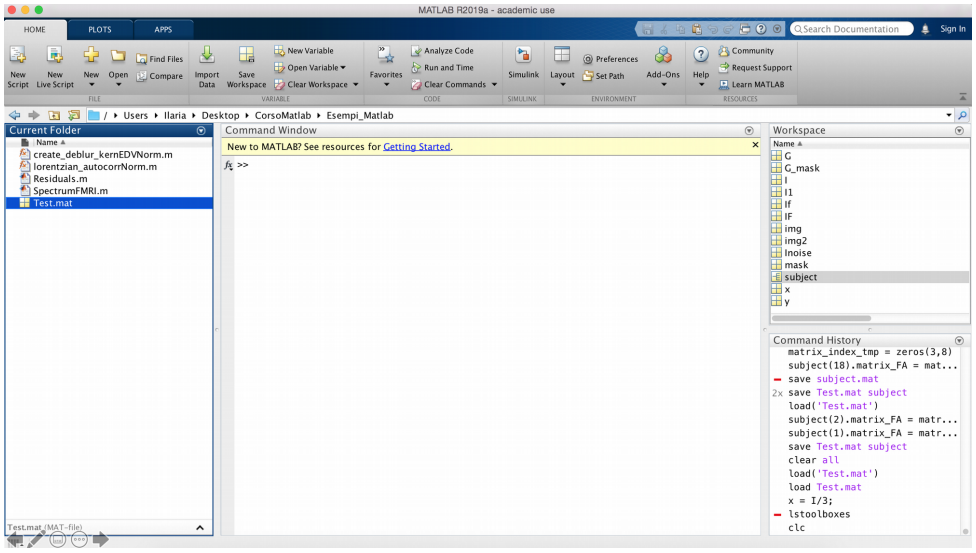


Figura 4: layout di Matlab

Il layout di Matlab è costituito da 4 sezioni principali:

- **Command Window**: finestra principale di interazione e contiene il comando prompt(>>).

  E' la finestra che ci permette di interagire con Matlab: serve oer inserire comandi e stampare a video i risultati.

- **Command History**: visualizza una lista di comandi precendetemente digitati.

  La cronologia rimane per più sessione, i comandi possono essere caricati direttamente sulla command window ed eventualmente modificati oppure doppio click per rieseguirli

- **Workspace**: contiene la lista di tutte le variabili create durante la sessione di lavoro corrente, insieme alle loro informazioni principali

- **Current Folder**: mostra i file che sono contenuti nella cartella di lavoro corrente, anche suddivisi per tipologia

Matlab può essere utilizzato per eseguire calcoli di base (operazioni aritmetiche, esponenziali e numeri complessi) come una normale calcolatrice scientifica.

Definizione di matrici e vettori

Matlab è un linguaggio di programmazione orientato alle matrici.

Gli **scalari** sono rappresentati come matrici 1×1.

I **vettori** sono particolari metrici con una sola riga e n colonne (*vettori riga*) o con una sola colonna e m righe (*vettori colonna*).

Matlab alloca utomaticamente le matrici in memoria pertanto non è necessario dichiararne le dimensioni prima di usarle.

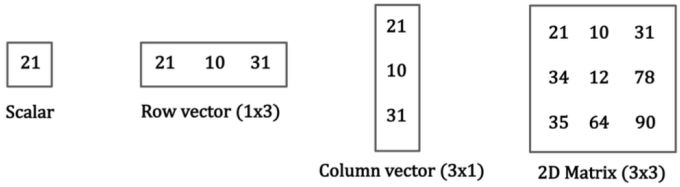


Figura 5: tipologie di matrici in Matlab

Le matrici si inserisco come segue:

- gli elementi di una colonna sono separati da spazi o virgole
- le righe sono separate da punto e virgola (;)
- le matrici si racchiudono tra parentesi quadre []

Per accedere agli elementi di una matrice si usano gli indici di riga e colonna racchiusi tra parentesi tonde ():

>> **x(3)** restituisce il terzo elemento del vettore x

>> **A(2,3)** restituisce l'elemento alla seconda riga e terza colonna della matrice A

>> **A(1,:)** restituisce la seconda riga della matrice A

>> **A(:,1)** restituisce la prima colonna della matrice A

>> **A(2:3,1:2)** estrae una sottomatrice di A costituita dagli elementi delle righe 1 e 2 e delle colonne 2 e 3

Le matrici possono esser costruite a partire da matrici di dimensioni minori:

>> **x(5) = abs(x(1))** produce x = [-1.3000 2.6667 4.8000 0 1.3000]

La dimensione di x viene automaticamente incrementata per inglobare il nuovo elemento e gli elementi indefinti viene posto uguale a zero.

Per creare un vettore di punti equispaziati si può usare l'istruzione x = [inizio:passo:fine]:

>> **x = [1:4**] produce x = [1 2 3 4]

>> **x = [1:3:8**] produce y = [1 4 7]

In alternativa si può usare l'istruzione `linspace(inizio, fine, num_punti)`, se si vuole un vettore di numri linearmente spaziati, altrimenti il comando `logspace(inizio, fine, num_punti)` per vettori con spaziatura logaritmica:

>> **z = linspace(2,14,6)** produce z = [2.0000 4.4000 6.8000 9.2000 11.6000 14.0000]

>> **k = logspace(1,2,7)** produce k = [10.0000 14.6780 21.5443 31.6228 46.4159 68.1292 100.0000]

In Matlab, sono implementate varie funzioni che permettono di definire in maniera semplice delle matrici particolari:

>> **zeros(m,n)** crea una matrice m x n di zeri

>> **ones(m,n)** crea una matrice m x n di uni

>> **eye(n)** crea una matrice identità di dimensione n x n

>> **rand(m,n)** crea una matrice m x n di numeri casuali uniformemente distribuiti nell'intervallo (0,1)

>> **randn(m,n)** crea una matrice m x n di numeri casuali con distribuzione normale standard (media 0 e varianza 1)

>> **diag(v)** crea una matrice diagonale con gli elementi del vettore v sulla diagonale principale

Per verificare la dimensione di una matrice o il numero di elementi inclusi, ci sono diverse funzioni:

>> **size(A)** restituisce la dimensione della matrice A

>> **length(A)** restituisce la lunghezza del vettore A

>> **numel(A)** restituisce il numero totale di elementi della matrice A

Operazioni tra matrici

Le operazioni fondamentali tra matrici, che devono avere la stessa dimensione per poter essere eseguite, sono:

`+` somma tra matrici

`-` sottrazione tra matrici

`.*` moltiplicazione elemento per elemento

`./` divisione elemento per elemento

`.^` elevamento a potenza elemento per elemento

Per valutare un'operazione "termine a termine", come molteplicare ogni termine del vettore A per ogni termine del vettore B, si usa l'operatore `.*`:

>> **A = [1 3 2]**

>> **B = [-1 2 1]**

>> **C = A .* B**

C = [-1 6 2]

Lo stesso si applica anche ad altre operazioni come la divisione e l'elevamento a potenza.

L'operazione di *trasposizione* è eseguita con l'operatore ':

>> A = [1 2 3; 4 5 6]

>> B = A'

B = [1 4; 2 5; 3 6]

Altre operazioni tra matrici imporatanti sono:

>> **inv(A)** calcolo dell'inversa della matrice A (quadrata)

>> **det(A)** calcolo del determinante della matrice A (quadrata)

>> **diag(A)** estrae la diagonale principale della matrice A come vettore

>> **trace(A)** calcola la somma degli elementi della diagonale principale della matrice A (quadrata)

>> **eig(A)** calcola gli autovalori e gli autovettori della matrice A (quadrata)

>> **rank(A)** calcola il rango della matrice A

>> **norm(A)** calcola la norma della matrice A

>> **sum(A)** calcola la somma degli elementi di ogni colonna della matrice A

>> **prod(A)** calcola il prodotto degli elementi di ogni colonna della matrice A

>> **min(A)** calcola il minimo degli elementi di ogni colonna della matrice A

>> **max(A)** calcola il massimo degli elementi di ogni colonna della matrice A

>> **mean(A)** calcola la media degli elementi di ogni colonna della matrice A

Comandi utili e istruzioni dos-like

In Matlab, ci sono una serie di funzioni di base che possono essere d'aiuto nella programmazione:

>> **clc** pulisce la Command Window

>> **dir** elenca i file nella cartella corrente

>> **pwd** mostra il percorso della cartella corrente

Altre istruzioni utili sono:

>> **type filename** visualizza il contenuto del file filename.m

>> **delete filename** elimina il file filename.m

>> **save filename** salva le variabili del workspace nella cartella corrente nel file filename.mat

>> **load filename** carica le variabili dal file filename.mat nel workspace

>> **! command** esegue il comando di sistema command (es. `!dir` elenca i file nella cartella corrente)

>> ↑ richiama l'ultimo comando eseguito nella Command Window

Inoltre, per ottenere aiuto su una funzione o un comando specifico, si può utilizzare:

>> **help functionname** visualizza la documentazione della funzione functionname

>> **doc functionname** apre la documentazione completa della funzione functionname in una finestra separata

>> **lookfor keyword** cerca la keyword nella documentazione di tutte le funzioni di Matlab

Script e function

Invece di inserire comandi direttamente nella Command Window, è possibile creare dei file di testo chiamati **script** o **function** che contengono una serie di comandi Matlab (estensione `.m`).

Gli *script file* contengono una serie di comandi Matlab che vengono eseguiti in sequenza quando lo script viene richiamato.

Gli script condividono lo stesso workspace della Command Window, quindi possono accedere e modificare le variabili definite nella Command Window.

Le *function file* sono simili agli script, ma hanno una struttura più formale.

Le variabili vengono passate per valore, quindi contengono vari comandi che necessitano di variabili di input per essere eseguiti e producono variabili di output:

```
function [output] = functionName(input)
    % Corpo della function
end
```

Le variabili definite all'interno di una function sono locali a quella function e non influenzano il workspace della Command Window.

Esempio di function che calcola una gaussiana:

```
function g1 = guassian(alpha, t)
    g1 = exp(-alpha*t.^2);
    ... command
end
```

I commenti in Matlab iniziano con il "%" e durano fino a afine della riga.

Un doppio commento ha un significato particolare: delimita l'inizio di un blocco di codice che può essere eseguito singolarmente all'interno di uno script o di una function.

E' una caratteristica utile per testare porzioni di codice senza dover eseguire l'intero script o function.

Istruzioni di controllo

Tra le istruzioni di controllo che ci possono essere utili in Matlab

`if ... elseif ... else ... end` : istruzione condizionale

```
if condizione1
    ... comandi se condizione1 è vera
elseif condizione2
    ... comandi se condizione2 è vera
else
    ... comandi se nessuna condizione è vera
end
```

`for ... end` : ciclo for

```
for indice = inizio:passo:fine
    ... comandi da eseguire in ogni iterazione
end
```

`while ... end` : ciclo while

```
while condizione
    ... comandi da eseguire finché la condizione è vera
end
```

Gli operatori logici principali in Matlab sono:

`&` : operatore AND logico

`|` : operatore OR logico

: operatore NOT logico

`xor` : operatore XOR logico

Gli operatori relazionali principali in Matlab sono:

`==` : uguale a

`=` : diverso da

`>` : maggiore di

`<` : minore di

`>=` : maggiore o uguale a

`<=` : minore o uguale a